Ejercicio Demostrativo

Auxiliar: xxxx

Tiempo estimado: 20 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Estructura de Datos - xx]

1. Marco Formativo

1.1 Valores

Precisión	Un solo error al calcular el índice cambia el lugar donde se guarda una clave y hace que después no se encuentre. El estudiante que verifica cada operación módulo y cada sondeo logra dominar la estructura.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Maneja grandes volúmenes de datos aplicando tablas hash para optimizar búsquedas.
Competencia Específica	Calcula índices con una función hash de módulo; resuelve colisiones mediante encadenamiento y sondeo lineal; ejecuta búsqueda y eliminación (con marca de eliminado); y analiza el efecto del factor de carga en el rendimiento.

2. Palabras Clave

Función hash: Función que convierte una clave en un índice del arreglo. Con claves enteras la más común es h(k) = k mod m, donde m es el tamaño de la tabla.

Colisión: Situación en la que dos claves distintas producen el mismo índice.

Encadenamiento: Método de resolución de colisiones donde cada posición guarda una lista enlazada con todas las claves que comparten el mismo índice.

Sondeo lineal: Método de direccionamiento abierto: si la posición está ocupada se prueba la siguiente, (i + 1) mod m, hasta hallar una libre. Puede producir agrupamiento primario.

Factor de carga (α): Relación n / m entre claves almacenadas y tamaño de la tabla. Cuanto más cerca de 1, más colisiones y sondeos; al superar un umbral se redimensiona (rehashing).

3. Resumen

Esta sesión introduce la tabla hash, una estructura que almacena pares clave–valor y accede a ellos calculando su posición con una función hash. El estudiante ya conoce arreglos, listas y árboles; ahora compara su rendimiento con el de un AVL en búsqueda, inserción y eliminación.

La clase se orienta a que el estudiante calcule índices con la función módulo, resuelva colisiones con encadenamiento y sondeo lineal, y comprenda el efecto del factor de carga.

4.Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

El ejercicio consiste en construir una tabla hash de tamaño 7 con la función `h(k) = k mod 7` e insertar la secuencia `[10, 22, 31, 4, 15, 17]`. Primero se resuelven las colisiones con encadenamiento y luego con sondeo lineal; al final se busca la clave 17 y se elimina la clave 31, observando por qué la eliminación en sondeo lineal requiere una marca de eliminado.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

> Demostrar cómo una tabla hash almacena y busca claves mediante una función hash, y cómo se resuelven las colisiones con encadenamiento y sondeo lineal, con el fin de que el estudiante comprenda que el rendimiento O(1) depende de una buena distribución de las claves y de mantener un factor de carga bajo.

4.3 RECURSOS NECESARIOS

- **Software:** Java JDK 11+, IDE (IntelliJ IDEA / Eclipse / VS Code)
- **Herramientas técnicas:** Compilador javac, terminal/consola
- **Materiales de apoyo:** Diapositivas de la semana, pizarrón, visualizador online https://visualgo.net/en/hashtable
- **Archivos:** Archivo base `TablaHash.java` con el arreglo y la función hash ya implementados
- **Configuración mínima:** Java 11+, 4 GB RAM, proyector

4.4 DESARROLLO PASO A PASO

Paso 1 — Función hash y tabla vacía (5 min)

Dibujar en el pizarrón una tabla de tamaño `m = 7` (posiciones 0 a 6) y usar `h(k) = k mod 7`. Pedir a los estudiantes que calculen el índice de `[10, 22, 31, 4, 15, 17]` sin que lo calcule el auxiliar:

- **Índices esperados:** `10→3  22→1  31→3  4→4  15→1  17→3`
- **Claves que colisionan:** `31, 15 y 17`
- **Índice más cargado:** `3 (10, 31 y 17)`

Qué observar: Tres de las seis claves caen en una posición ya ocupada: la función hash no garantiza índices únicos, por eso se necesita una estrategia para resolver colisiones.

Paso 2 — Colisiones con encadenamiento (5 min)

Insertar las seis claves en orden; cada posición guarda una lista enlazada:

java

```
void insertar(int clave) {
    int i = clave % capacidad;
    tabla[i].agregarAlFinal(clave); // lista enlazada
}
```

```
Estado final de la tabla:
[0]  vacío
[1]  22 -> 15
[2]  vacío
[3]  10 -> 31 -> 17
[4]  4
[5]  vacío
[6]  vacío
```

Qué observar: El índice 3 acumula tres claves, por lo que buscar 17 recorre 3 nodos. El factor de carga es α = n / m = 6 / 7 ≈ 0.86.

Paso 3 — Colisiones con sondeo lineal (5 min)

Vaciar la tabla y reinsertar las mismas claves; si la posición está ocupada se prueba la siguiente, `(i + 1) mod m`:

java

```
void insertar(int clave) {
    int i = clave % capacidad;
    while (tabla[i] != VACIO)
        i = (i + 1) % capacidad; // siguiente posición
    tabla[i] = clave;
}
```

Procedimiento:

1. 10 → índice 3 (libre) y 22 → índice 1 (libre).
2. 31 → índice 3 ocupado, prueba el 4 → queda en **4**. Luego 4 → índice 4 ocupado → queda en **5**.
3. 15 → índice 1 ocupado → queda en **2**. Luego 17 → índices 3, 4 y 5 ocupados → queda en **6**.

Resultado: La tabla queda así:

```
Índice:  [0]  [1]  [2]  [3]  [4]  [5]  [6]
Clave:    -    22   15   10   31    4   17
```

Qué observar: Las claves forman un bloque contiguo de la posición 1 a la 6 (agrupamiento primario): mientras más grande el bloque, más sondeos necesita cada nueva inserción.

Paso 4 — Búsqueda y eliminación (5 min)

Sobre la tabla con sondeo lineal, buscar la clave **17** y luego eliminar la clave **31**:

Procedimiento:

1. Buscar 17: índice 3 → 4 → 5 → 6, se encuentra en el **cuarto sondeo** (con encadenamiento fueron 3 comparaciones).
2. Eliminar 31 (índice 4): si la casilla se deja vacía, la búsqueda de 17 se detendría en el índice 4 y diría que no existe.
3. Solución: marcar la casilla como **eliminado** (lápida). La búsqueda continúa sobre ella y una inserción puede reutilizarla.

Resultado: La búsqueda de 17 sigue funcionando porque la casilla 4 conserva la marca de eliminado.

Qué observar: Eliminar en sondeo lineal no es solo borrar el valor: hay que preservar la cadena de sondeo.

Observaciones Importantes

- Una buena función hash reparte las claves de manera uniforme; usar un tamaño m primo (como 7) ayuda a reducir las colisiones.
- Con encadenamiento el factor de carga puede superar 1; con sondeo lineal debe mantenerse bajo 1 (se recomienda menos de 0.7) y se redimensiona la tabla (rehashing) al superarlo.
- La tabla hash no guarda orden: para recorrer las claves ordenadas se necesita ordenarlas aparte o usar un árbol como el AVL.

4.5 Tabla Comparativa — Tabla Hash vs AVL

TABLA COMPARATIVA

Operación	Hash (promedio)	Hash (peor caso)	AVL (siempre)	Observación
Buscar	O(1)	O(n)	O(log n)	Hash es más rápido en promedio
Insertar	O(1)	O(n)	O(log n)	El peor caso ocurre si todo colisiona
Eliminar	O(1)	O(n)	O(log n)	Con sondeo lineal requiere marca
Recorrido ordenado	No ordena	No ordena	O(n)	Ventaja clave del AVL
Redimensionar	O(n)	O(n)	No aplica	Rehashing al superar el factor de carga

4.6 Resultado Esperado

Operación	Resultado
Insertar `[10,22,31,4,15,17]` con encadenamiento	Lista en el índice 3: 10, 31, 17. Lista en el índice 1: 22, 15.
Insertar las mismas claves con sondeo lineal	Posiciones 0 a 6: `[-,22,15,10,31,4,17]`
Buscar 17 con encadenamiento	Se encuentra tras 3 comparaciones (10, 31, 17).
Buscar 17 con sondeo lineal	Se encuentra tras 4 sondeos (índices 3, 4, 5 y 6).
Eliminar 31 con sondeo lineal	Casilla 4 queda marcada como eliminado; 17 sigue encontrándose.
Factor de carga de la tabla	$\alpha = 6 / 7 \approx 0.86$

4.7 Aplicación Práctica

Las tablas hash son la base de los diccionarios y mapas de casi todos los lenguajes (HashMap en Java, dict en Python) y de los índices hash en bases de datos: permiten localizar un registro por su clave en tiempo constante promedio, sin recorrer toda la tabla.

En el proyecto del curso, una tabla hash indexada por código permite validar en O(1) si un registro ya existe antes de insertarlo en la estructura principal, evitando búsquedas lineales o recorridos completos del árbol.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Los estudiantes olvidan aplicar el módulo con el tamaño de la tabla y calculan mal el índice. Hay que pedirles que escriban siempre la operación k mod m antes de ubicar la clave.
- En sondeo lineal, al eliminar dejan la casilla vacía y luego la búsqueda se detiene antes de tiempo, concluyendo que la clave no existe.

Recomendaciones:

- Dibujar la tabla vacía en el pizarrón antes de empezar y usar un color distinto para cada colisión resuelta, así se ve de dónde viene cada desplazamiento.
- Mostrar en el visualizador online (visualgo.net) la inserción con colisiones y la eliminación para que el estudiante vea la animación de cada sondeo.

Puntos de mayor dificultad:

- Entender por qué eliminar en sondeo lineal requiere una marca de eliminado y no basta con vaciar la casilla.
- Relacionar el factor de carga con el rendimiento y con el momento en que conviene redimensionar la tabla.

6. Referencias

1. GeeksforGeeks. (s. f.). Hashing Data Structure. https://www.geeksforgeeks.org/hashing-data-structure/
2. Visualizador interactivo de tablas hash: https://visualgo.net/en/hashtable
3. Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Cap. 11: Hash Tables.
4. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson. Cap. 5: Hashing.
5. Material de cátedra: Diapositivas EDD — Tablas Hash. USAC, 2026.